

JDBC basics

COMPLETE LEARNING GUIDE

What You'll Learn

JDBC (Java Database Connectivity) is the standard API Java uses to connect to and interact with relational databases, and understanding its core objects — Driver, Connection, Statement/PreparedStatement, and ResultSet — is essential both for interviews and for any real backend work, since even ORM frameworks like Hibernate or Spring Data ultimately compile down to JDBC calls underneath. This guide covers how a JDBC connection is established, why PreparedStatement must be used instead of Statement for any query involving user input (SQL injection prevention, not just convenience), how to iterate a ResultSet correctly, resource management for connections and statements via try-with-resources, and the fundamentals of manual transaction control with commit/rollback. This is foundational knowledge for the capstone application later in the course, which will very likely need to persist data to a real database.

1. JDBC Architecture: Driver, Connection, Statement, ResultSet

A JDBC Driver is a database-specific implementation (loaded automatically via Java's ServiceLoader mechanism since JDBC 4.0) that translates the standard JDBC API calls into that specific database's wire protocol — this is what lets application code stay nearly identical whether it's talking to PostgreSQL, MySQL, or Oracle, with only the connection URL and driver dependency changing. A Connection represents a single session with the database; a Statement (or its safer subtype, PreparedStatement) represents a SQL command sent over that connection; a ResultSet represents the tabular data returned by a query, which must be read while the connection remains open.

2. Establishing a Connection

DriverManager.getConnection(url, username, password) opens a Connection given a JDBC URL that encodes the database type, host, port, and database name (e.g. jdbc:postgresql://localhost:5432/mydb) — the driver is picked automatically based on the URL's prefix, as long as the corresponding driver JAR is on the classpath. Connections are relatively expensive to establish (network handshake, authentication) so production applications virtually never call DriverManager.getConnection() per request — they draw connections from a pool (covered below), but the underlying API is identical either way.

JAVA

```
String url = "jdbc:postgresql://localhost:5432/company_db";
try (Connection conn = DriverManager.getConnection(url, "app_user", "secret")) {
    // use the connection
} catch (SQLException e) {
    System.err.println("Connection failed: " + e.getMessage());
}
```

3. PreparedStatement vs. Statement: SQL Injection Prevention

Statement executes a raw SQL string as-is, which means building a query by concatenating user input directly into the SQL text ("SELECT * FROM users WHERE name = " + userInput + "'") lets an attacker supply input like ' OR '1'='1 to alter the query's logic entirely — this is SQL injection, one of the most common and most damaging web application vulnerabilities. PreparedStatement pre-compiles the SQL with ? placeholders for parameters, and values are bound separately via typed setter methods (setString, setInt); the database driver ensures bound values are always treated as literal data, never as executable SQL syntax, which eliminates injection for parameterized values entirely. PreparedStatement also improves performance for repeated queries, since the database can cache the compiled query plan.

JAVA

```
// VULNERABLE -- never do this with user input
String bad = "SELECT * FROM users WHERE email = '" + userInput + "'";

// SAFE -- parameters are bound, never interpreted as SQL
String sql = "SELECT id, name, email FROM users WHERE email = ?";
try (PreparedStatement stmt = conn.prepareStatement(sql)) {
    stmt.setString(1, userInput);    // 1-indexed, not 0-indexed
    try (ResultSet rs = stmt.executeQuery()) {
        while (rs.next()) {
            System.out.println(rs.getInt("id") + ": " + rs.getString("name"));
        }
    }
}
}
```

4. Iterating a ResultSet Correctly

ResultSet.next() advances the cursor to the next row and returns false once there are no more rows, so the standard iteration pattern is a while (rs.next()) loop — calling a getter (getString, getInt) before the first next() call, or after next() has returned false, throws a SQLException because the cursor isn't positioned on a valid row. Columns can be retrieved by 1-indexed position (rs.getString(1)) or by column name (rs.getString("email")); using column names is generally preferred for readability and resilience to a query's column order changing, at a very small performance cost from the name-to-index lookup.

5. Resource Management: try-with-resources for JDBC

Connection, Statement/PreparedStatement, and ResultSet all implement AutoCloseable, and all three should be wrapped in try-with-resources (nested, since a PreparedStatement is created from a Connection and a ResultSet from a Statement) to guarantee they're released even when an exception occurs mid-query — leaked database connections are a classic production incident, since connection pools have a finite size and a leak eventually exhausts it, causing every subsequent request to hang waiting for a connection that never becomes available.

6. Transactions: commit, rollback, and autocommit

By default, a JDBC Connection runs in autocommit mode, where every individual statement is committed to the database immediately as its own transaction — fine for single, independent statements, but wrong whenever multiple statements must succeed or fail together as one atomic unit (like transferring money: debit one account, credit another). Calling `conn.setAutoCommit(false)` starts manual transaction control: subsequent statements are staged but not permanent until `conn.commit()` is called explicitly, and if any statement fails partway through, `conn.rollback()` undoes every staged change back to the transaction's start, preventing a half-completed operation from corrupting data.

JAVA

```
try (Connection conn = DriverManager.getConnection(url, user, pass)) {
    conn.setAutoCommit(false);
    try {
        debit(conn, fromAccountId, amount);
        credit(conn, toAccountId, amount);
        conn.commit();           // both succeed -- make permanent
    } catch (SQLException e) {
        conn.rollback();         // either failed -- undo both
        throw e;
    }
}
```

7. Connection Pooling: Why Raw DriverManager Isn't Used in Production

Opening a fresh physical connection per request is expensive (TCP handshake, authentication, session setup) and doesn't scale, so production applications use a connection pool (HikariCP is the modern default, especially in Spring Boot) that maintains a set of already-open connections and hands them out on request, returning them to the pool (not actually closing the physical connection) when the application code calls `close()`. This means the application code's usage pattern — open with try-with-resources, use, close — stays identical whether connections come from DriverManager directly or from a pool; only the configuration changes, which is one of the reasons JDBC's basic API is worth learning even though production code rarely calls `DriverManager.getConnection()` directly.

Common Pitfalls

- Building SQL by string concatenation with user input instead of using PreparedStatement with bound parameters, opening the application to SQL injection.
- Forgetting PreparedStatement parameters are 1-indexed, not 0-indexed, causing an off-by-one binding error.
- Calling a ResultSet getter before the first rs.next() (or after it returns false), throwing a SQLException for an invalid cursor position.
- Not wrapping Connection/Statement/ResultSet in try-with-resources, leaking connections that eventually exhaust a connection pool under load.
- Leaving autocommit on for a multi-statement operation that must be atomic, risking a half-completed update if a later statement fails.
- Forgetting to call conn.rollback() in the catch block of a manual transaction, leaving a failed transaction's partial changes staged indefinitely.
- Calling DriverManager.getConnection() directly in production request-handling code instead of drawing from a connection pool, incurring a full connection setup cost on every request.

Key Takeaways

- Always use PreparedStatement with bound parameters for any query involving external input — string-concatenated SQL is a direct SQL injection vulnerability.
- ResultSet.next() must be called before reading any column, and iteration is a standard while (rs.next()) loop.
- Connection, Statement, and ResultSet are all AutoCloseable — wrap them in (nested) try-with-resources to avoid leaking database resources.
- Disable autocommit and use explicit commit()/rollback() whenever multiple statements must succeed or fail together as one atomic transaction.
- Production applications draw connections from a pool (e.g. HikariCP) rather than opening a fresh DriverManager connection per request, but the application-level JDBC API usage is identical either way.

Learn More

Oracle JDBC Basics Tutorial

<https://docs.oracle.com/javase/tutorial/jdbc/basics/index.html>

Oracle Java SE Docs — PreparedStatement

<https://docs.oracle.com/en/java/javase/17/docs/api/java.sql/java/sql/PreparedStatement.html>

OWASP — SQL Injection Prevention Cheat Sheet

https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html